

Este documento detalla los componentes de infraestructura físicos y lógicos necesarios para soportar el funcionamiento del API Gateway en producción, incluyendo servidores, servicios de red, almacenes de estado, herramientas de monitorización y APIs de terceros.

El Gateway está diseñado para ejecutarse de forma aislada en contenedores ligeros de Linux, delegando estados distribuidos y recolección de telemetría a componentes externos específicos.

Recurso	Imagen / Proveedor	Rol / Propósito
Criticidad / Modo Offline		
API Gateway	node:20-alpine	Motor del proxy inverso, validación JWT y enrutamiento.
Bloqueante / N/A	Redis Cache	redis:7-alpine
Almacenamiento en memoria de contadores de Rate Limiting.	Alta / Permitido (Fail-Open)	Prometheus
prom/prometheus:v3.4.1	Recolector de series de tiempo para telemetría (/metrics).	Opcional / Permitido
Grafana	grafana/grafana:12.1.0	Panel visual de analíticas para monitoreo en tiempo real.
Opcional / Permitido	Dozzle	amir20/dozzle:latest
Consola web ligera para inspección de logs en caliente.	Opcional / Permitido	

El Gateway enruta y asegura peticiones dirigidas a los siguientes backends ascendentes (upstreams):

Servicio Upstream	Dirección de Target	Rol y Tipo de Enlace
Criticidad		
Mock Backend	http://mock-service:8080	Servicio mock interno (Enrutamiento DNS Docker)
Opcional (Pruebas)	PokéAPI	https://pokeapi.co/api/v2
API externa pública (Enrutamiento HTTPS WAN)	Opcional (Ruta \api)	Httpbin
https://httpbin.org	API externa de test HTTP (Enrutamiento HTTPS WAN)	Opcional (Ruta \httpbin)

Para mitigar fallos en cascada causados por redes inestables o sobrecarga en los servicios del backend, el API Gateway implementa tres mecanismos clave a nivel de infraestructura:

1. Degradación de Rate Limiting (Hot Redis Fail-Open)

El Gateway se conecta a Redis para coordinar cuotas de solicitudes. Sin embargo, para evitar que una interrupción en la instancia de Redis paralice toda la infraestructura:

- Se evalúa la directiva `redis.onFailure: open`.
- En caso de caída del servidor Redis, las solicitudes del usuario no se bloquean. El middleware de limitación de tasa se degrada de manera silenciosa (fail-open) y el tráfico continúa fluyendo de forma segura.

2. Aislamiento de Fallos mediante Circuit Breaker

Cada ruta configurada cuenta con un Circuit Breaker a nivel de cliente de red.

- Si un backend externo (como PokéAPI) comienza a devolver errores repetitivos o latencias elevadas, el circuito cambia al estado **Abierto**.
- En estado abierto, el Gateway devuelve un error local controlado en milisegundos (`503 Service Unavailable`), protegiendo los recursos locales (sockets, CPU, RAM) al no intentar conexiones inútiles contra servidores caídos.

3. Connection Pooling y Reuso de Sockets con Undici

En lugar de crear y destruir una conexión TCP/TLS por cada solicitud, el motor de proxy utiliza **Undici Connection Pools**.

- Mantiene pools de sockets abiertos y persistentes (keep-alive) por cada destino de target diferenciado.
- Limita dinámicamente el número máximo de conexiones concurrentes y gestiona colas de peticiones pendientes de socket.